

SAPIENZA
UNIVERSITÀ DI ROMA

Machine Learning for Sensor Signals: Time Series Classification in Real-World Environments

Faculty of Mathematical, Physical and Natural Sciences
Bachelor's Degree in Mathematical Sciences for Artificial Intelligence (L-35)

Michele Magrini
ID number 2066963

Advisor
Marco Sciandrone

Academic Year 2024/2025

Abstract

This thesis addresses the problem of automatic classification of multivariate sensor signals, collected by devices developed by Sensichips S.r.l., for the identification of 17 different chemical substances. Four distinct approaches were explored: classification at individual time instants, within fixed-length time windows, using compact representations via Radial Basis Functions (RBF), and through continuous sequential processing with recurrent neural networks (RNN). For each approach, both neural and classical models (Random Forest) were designed and compared, evaluating the trade-off between accuracy, memory footprint, and robustness. Experimental results show that the time window approach provides the best balance between performance and complexity, while RNNs emerge as a promising alternative for continuous temporal processing. The full codebase, including preprocessing, training, and evaluation scripts, is publicly available to ensure reproducibility and support future extensions.

Machine Learning for Sensor Signals: Time Series Classification in Real-World Environments

Sapienza University of Rome – Faculty of Mathematical, Physical and Natural Sciences
Author: Michele Magrini (magrini.2066963@studenti.uniroma1.it)
Advisor: Marco Sciandrone (marco.sciandrone@uniroma1.it)
Sensichips S.R.L. (sensichips.com)

Contents

Introduction **1**

1 The SCA Sensor Dataset **3**
1.1 Dataset overview . . . 3
1.2 Data acquisition procedure . . . 3
1.3 Filtering and initial preparation . . . 4
1.4 Statistical and visual exploration of the dataset . . . 5
1.5 Real-world considerations . . . 6

2 Classification at Individual Time Instants **8**
2.1 Multilayer Perceptron . . . 8
2.1.1 Model operation and learning . . . 8
2.1.2 The Common MLP Block . . . 10
2.2 Encoder MLP . . . 10
2.3 Settings shared by all models . . . 12
2.4 Results and observations . . . 12
2.4.1 Metric definitions . . . 12

3 Classification with Time Windows **15**
3.1 Data structure: time windows . . . 15
3.2 Column Selective Encoder MLP . . . 16
3.3 Convolutional Neural Networks . . . 17
3.4 Results and observations . . . 18

4 Temporal Analysis of Predictions **21**
4.1 Motivation and purpose . . . 21
4.2 Results and observations . . . 21

5 Binary Classification Experiments **23**
5.1 Binary classification at individual time instants . . . 24
5.2 Binary classification with time windows . . . 24
5.3 Observations . . . 25

6 Classification with RBF Interpolation **26**
6.1 Approach . . . 26
6.2 Input data structure . . . 26
6.3 Random Forest . . . 27

6.4 Results and observations 27

7 Classification with Continuous Temporal Processing 29
7.1 Recurrent Neural Networks 29
7.2 Gated Recurrent Unit Networks 30
7.3 Long Short-Term Memory Networks 30
7.4 Classification of Complete Variable-Length Time Series 31
7.4.1 Results and Observations 31
7.5 Applying the RNN to Time Windows 31
7.5.1 Results and Observations 31

8 Comparative Discussion of the Models 34

Conclusions 36

Bibliography 37

List of Figures

1.1 Distribution of experiments (left) and samples (right), grouped by label and colored according to the training, validation, or test split. . 5
1.2 Visual and statistical analysis of the dataset. Upper left: correlation matrix among the 16 features. Upper right: 3D projection of the data through PCA. Bottom: absolute feature contributions to the first three principal components (PC1, PC2, and PC3). 6

2.1 Comparison between the simple **MLP** (left) and the **Encoder MLP** (right). The latter introduces a latent space (green) before the *Common MLP Block*. 11
2.2 Learning curve of the time-instant MLP during training: loss and accuracy on the training and validation sets. 13
2.3 Confusion matrices on the test set for the time-instant MLP. Left: *sample-level* classification; right: *experiment-level* classification. . . . 14

3.1 Example representation of a 100×16 time-window sample (time steps $\times$ features). Structural differences among columns are clearly visible, indicating the heterogeneous response of the features to the chemical signal. 15
3.2 Architecture of the **CSEMLP**. The signal is processed by a column-selective block (CS) and a fully connected block (L) before entering the *Common MLP Block*. 17
3.3 Architecture of the **CNN1D**. The 100×16 time sequence is transposed into a 16×100 tensor and processed with 1D convolutions along the temporal axis. The output is then flattened and passed to the *Common MLP Block*. 17
3.4 Learning curve of EMLPs on time windows during training: loss and accuracy on the training and validation sets. 19
3.5 Confusion matrices on the test set for EMLPs on time windows. Left: *sample-level* classification; right: *experiment-level* classification. . . . 20

4.1 Temporal distribution of correct predictions for the two approaches. Left: MLP on individual time instants; right: EMLP on time windows. Each row represents an experiment, ordered by chemical class. . . . 22

6.1 Example RBF approximation for ammonium chloride. 27

7.1 Architecture of a generic RNN. Vector x_t connects to hidden state h_t, which in turn connects to h_{t-1} and h_{t+1}. The output layer communicates only with the final hidden state h_N. 29
7.2 Learning curves of the GRU model on variable-length time series. . . 32
7.3 Results of the GRU with the time-window approach. Left: sample-level and experiment-level confusion matrices; right: the associated temporal distributions, analogous to Figure 4.1. 32

8.1 Relationship between experiment-level accuracy and model memory footprint. 35

List of Tables

2.1 Model performance with the time-instant approach. 13

3.1 Performance of the time-window models in terms of sample-level accuracy (S_{acc}) and experiment-level accuracy (E_{acc}), as defined in Section 2.4.1. 18

5.1 Per-fold and mean performance of the MLP on the binary ETHANOL-versus-ACETONE task with the time-instant approach. 24
5.2 Per-fold and mean performance of EMLPs on the binary ETHANOL-versus-ACETONE task with the time-window approach. 24

6.1 Performance of the different models with the RBF approach. 28

7.1 Performance of GRU and LSTM models on complete sequences. . . . 31

Introduction

Over the past few decades, *Machine Learning* (ML) has become an essential tool for analyzing and interpreting large volumes of heterogeneous data. Among the many applications of ML techniques, the automatic classification of signals collected by sensors has become especially important. It is a complex yet crucial task in industrial[4], environmental[3], and biomedical[6;10] settings.

This thesis focuses on the development and evaluation of machine-learning models applied to multivariate time series obtained from chemical sensors. The data were provided through a collaboration with *Sensichips S.r.l.* and consist of a collection of experimental acquisitions, represented as matrices, recorded both under controlled conditions and in real-world environments.

Each experiment consists of a time sequence of measurements organized in 16-column tables. The columns correspond to different electrical characteristics measured by two distinct sensors operating at different frequencies and in different modes. Each experiment is associated with one of 17 possible chemical substances, which constitutes its classification label.

The aim of this work is to develop algorithms that can learn from historical data and correctly predict which substance is present from a new sequence of measurements. This is a supervised time-series classification problem, made more difficult by the noisy and variable nature of data collected in real-world environments and by variations in the duration and structure of the acquisitions.

To address this challenge, four distinct approaches were designed and implemented, each exploiting temporal information and data structure to a different extent.

- **Time-instant approach**: treats each row of measurements as an independent sample represented by a 16-dimensional vector. This simple strategy ignores temporal dependence between consecutive observations.
- **Time-window approach**: treats subsequences (windows) of temporally contiguous measurements as input units. Each fixed-size $N \times 16$ window allows the model to learn the local dynamics of the signal.
- **RBF approach**: approximates each column (time series) with a combination of radial basis functions, yielding a compact representation of the entire experiment. This representation reduces dimensionality and enables more stable classification.
- **Continuous temporal-processing approach**: uses recurrent models to process the entire temporal sequence of each experiment without dividing it

into windows. This approach allows the model to learn global representations by exploiting the complete temporal dependence in the data.

All approaches were evaluated according to their ability to classify the entire time series correctly. When a model does not analyze the whole time series, the final prediction is determined by majority voting over its partial predictions.

The contribution of this thesis is twofold: it implements and systematically compares different approaches, and it critically examines how real-world conditions affect modeling choices and results.

Chapter 1

The SCA Sensor Dataset

This chapter describes the dataset used to train and evaluate the classification models. Provided by *Sensichips S.r.l.*, it is a heterogeneous collection of experiments conducted with advanced chemical sensors under both real-world and controlled conditions. Understanding the structure of these data and how they were acquired is essential for motivating the modeling choices presented in the following chapters.

1.1 Dataset overview

The dataset, called the **SCA Sensor Dataset**, consists of numerous tables, each representing an individual **experiment**. Every experiment is a time sequence of measurements collected while the sensors were exposed to a specific chemical substance. Overall, the data cover 17 substance classes, including ethanol, acetone, ammonia, and other volatile compounds commonly used as targets for environmental sensors.

Each table contains 16 feature columns corresponding to electrical measurements acquired at different frequencies and in different operating modes. The number of rows varies from file to file and represents the temporal evolution of the signal: each row can be viewed as one instantaneous acquisition of all features. Labels do not vary by row; each file is associated in its entirety with a single class, that is, one substance.

1.2 Data acquisition procedure

The data were acquired using the *SCA* sensor [2], designed by Sensichips and based on two different sensing components:

- **SnO_2+Au sensor on a hotplate**: a tin-oxide sensor functionalized with gold nanoparticles. It was operated using **Temperature Cycled Operation (TCO)**, meaning that it was heated to six temperature levels between 150°C and 400°C. At each level, two in-phase measurements were acquired at 78.125 Hz and 200 Hz.
- **Al_2O_3 sensor on an interdigitated electrode**: an anodized-alumina sensor electrically stimulated at two voltage levels. This sensor uses **Voltage Cy-**

cled Operation (VCO); both the *in-phase* and *quadrature* components are recorded, but only at 78.125 Hz.

Measurements were performed either indoors, where the chemical compound was allowed to evaporate slowly and under controlled conditions, or in open environments subject to ambient turbulence.

1.3 Filtering and initial preparation

To ensure a minimum quality level for the data used by the classification models, a set of filtering criteria was applied. Only experiments satisfying both of the following conditions were considered valid:

- Each experiment had to contain at least **200 rows** (at least 200 temporal measurements), allowing the time-window approach to be applied with sufficient sliding positions.
- Each class (label) had to be represented by at least **10 valid experiments**, ensuring a minimum degree of variety and statistical significance within each group.

Dataset split: training, validation, and test

A key step in data preparation is splitting the dataset into three subsets: **training**, **validation**, and **test**. This separates model learning, hyperparameter optimization, and final assessment of the model's general performance.

For the dataset supplied by Sensichips, the split was performed at experiment level: all rows belonging to a given experiment are always assigned to the same subset. This choice is crucial to prevent temporal information or experiment-specific characteristics from being shared between training and test data, which would create a risk of *data leakage* and overestimated performance.

The data were divided according to the following proportions, calculated separately for each label:

- **Validation set**: 10% of the available experiments in each class, with a minimum of two experiments per label.
- **Test set**: 10% of the experiments in each class, again with a minimum of two experiments per label.
- **Training set**: all remaining experiments, used for the actual learning process.

This splitting strategy ensures balanced and consistent class representation at every stage while preserving the intrinsic structure of the data.

All multiclass models developed in this work were trained and evaluated using exactly the same dataset split. This consistency ensures a fair comparison among models by eliminating variation caused by different data partitions and preserving comparability across the reported results.

The numerical features were then normalized by centering them on the mean and scaling them by the standard deviation, both calculated exclusively on the training set. This operation harmonizes the scales of the different features and improves the numerical stability of the learning algorithms.

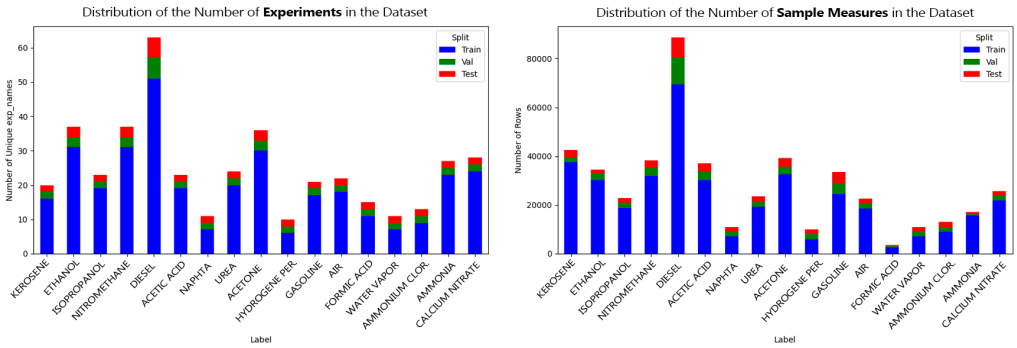


Figure 1.1 Distribution of experiments (left) and samples (right), grouped by label and colored according to the training, validation, or test split.

1.4 Statistical and visual exploration of the dataset

Visual and statistical exploration is essential for understanding the intrinsic structure of the data and guiding dimensionality-reduction or normalization choices. All plots in Figure 1.2 use the same feature ordering:

- The first 6: measurements acquired by the **SnO_2+Au** sensor at **200 Hz**, at temperatures from 150°C to 400°C.
- The next 6 (7–12): measurements from the same sensor at **78.125 Hz**, at the same temperatures.
- The final 4 (13–16): measurements from the **Al_2O_3** sensor at 78.125 Hz, divided into *in-phase* and *quadrature* components.

The first visualization (upper left in Figure 1.2) shows the sample-level **correlation matrix** among features. Measurements from the SnO_2+Au sensor at different temperatures are strongly correlated, suggesting that these channels measure the same phenomenon with different sensitivities. The four features associated with the Al_2O_3 sensor are likewise highly correlated.

This redundancy motivates the use of a **dimensionality-reduction** technique such as *Principal Component Analysis* (PCA). PCA is a linear transformation that projects the original data into a lower-dimensional space while maximizing variance along new principal directions. The components are mutually orthogonal and are constructed as linear combinations of the original features.

In the **3D PCA plot** (upper right in Figure 1.2), each point represents a time window projected onto the first three principal axes. The data form distinct "curves" for each substance that appear readily separable in the new space, indicating that PCA preserves the discriminative structure among classes well.

Finally, the lower portion of Figure 1.2 illustrates the absolute contribution of each feature to the first three components (PC1, PC2, and PC3). These plots were produced from the coefficients of the PCA **loading** matrix. Specifically, if $\mathbf{W} \in \mathbb{R}^{d\times 3}$ is the transformation matrix, each column represents a combination of the $d = 16$ original features.

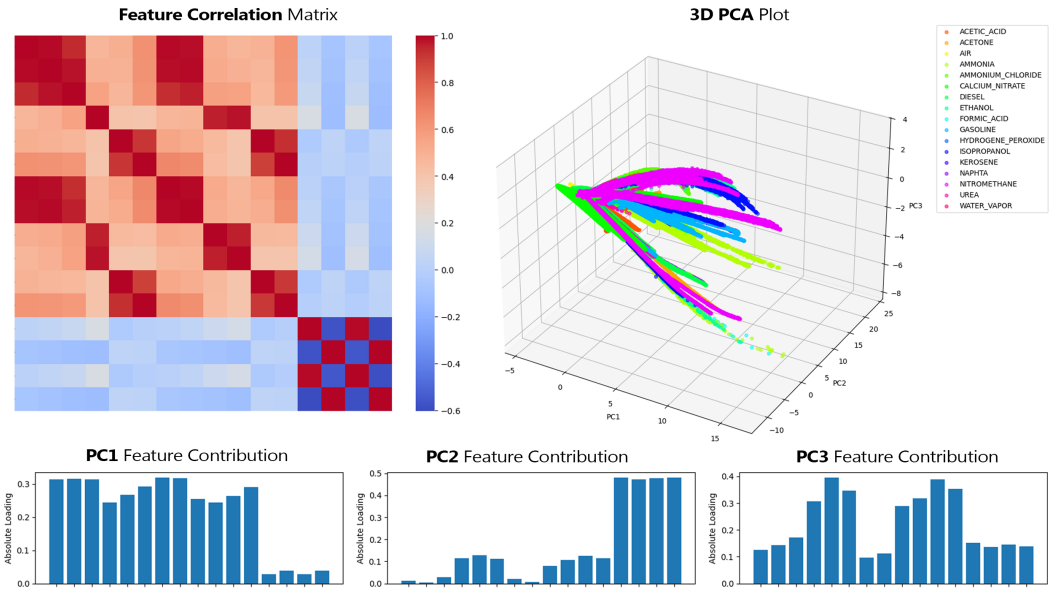


Figure 1.2 Visual and statistical analysis of the dataset. Upper left: correlation matrix among the 16 features. Upper right: 3D projection of the data through PCA. Bottom: absolute feature contributions to the first three principal components (PC1, PC2, and PC3).

The plots show that:

- **PC1** is dominated by features from the SnO_2+Au sensor, especially those measured at 200 Hz.
- **PC2** is influenced mainly by measurements from the Al_2O_3 sensor.
- **PC3** contains a mixture of significant contributions from both sensors.

This analysis confirms that much of the discriminative information is contained in only a few principal components, suggesting that compression and projection techniques may be useful when developing lightweight and efficient classification models.

1.5 Real-world considerations

One distinguishing feature of this dataset is that it originates from experiments conducted in real, rather than synthetic, environments. This introduces variability such as noise, temperature fluctuations, and contamination effects, making the classification task particularly interesting from a research perspective. As highlighted

by recent studies on environmental sensing[1;7], the robustness of ML algorithms in the presence of real-world noise is a key criterion for industrial applicability.

The presence of chemical classes with similar properties, such as ethanol and acetone, poses a further challenge because it requires the model to learn subtle yet meaningful representations.

Chapter 2

Classification at Individual Time Instants

This chapter examines an initial approach to the classification of sensor time series in which each time instant is considered independently. Although this strategy completely ignores temporal dependencies between consecutive measurements, it drastically reduces model complexity and provides a useful baseline for subsequent comparisons.

This technique is similar to approaches proposed in industrial and biomedical settings, where each measurement is treated as an independent observation [9]. It is often used when no clear temporal structure is available or when a lightweight model is required for real-time classification.

2.1 Multilayer Perceptron

The first model tested with this approach is a **Multilayer Perceptron (MLP)**, a *feed-forward* neural network composed of linear layers interleaved with nonlinear activation functions. The MLP is one of the fundamental architectures in deep learning and is widely used for classification and regression tasks [12].

Architecture

In our case, the model input is a vector $\mathbf{x} \in \mathbb{R}^{16}$, corresponding to a single time instant with 16 features. The model is structured as follows:

$$\mathbb{R}^{16} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{ReLU + Dropout}} \mathbb{R}^{64} \xrightarrow{\text{Linear}} \mathbb{R}^{17}$$

The final block from 64 to 17 neurons, also used in later models, is defined in Section 2.1.2 as the *Common MLP Block*.

2.1.1 Model operation and learning

During training, the MLP minimizes a loss function by comparing its predictions with the true labels. Weight optimization combines two key elements:

1. **Gradient descent**, which updates the weights in the direction that reduces the loss function;
2. The **backpropagation** algorithm, which computes gradients through the network layers.

Gradient descent

Given a loss function $\mathcal{L}(\theta)$, where θ represents the model parameters (weights and biases), gradient descent updates the parameters according to:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$$

where η is the *learning rate*, and $\nabla_\theta \mathcal{L}$ is the gradient of the loss with respect to the parameters. This method may be applied to the entire dataset (batch), to small subsets (mini-batches), or one sample at a time (stochastic).

Backpropagation

The gradient $\nabla_\theta \mathcal{L}$ is computed through *backpropagation*, a recursive application of the chain rule. If $z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$ is the linear input to layer l and $h^{(l)} = \sigma(z^{(l)})$ is the activated output, then the error at layer l is:

$$\delta^{(l)} = \left(\frac{\partial \mathcal{L}}{\partial h^{(l)}} \right) \cdot \sigma'(z^{(l)})$$

The gradient with respect to the weights is therefore:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (h^{(l-1)})^T$$

Optimization with Adam

In this work, we used the **Adam** optimizer (Adaptive Moment Estimation), an advanced variant of stochastic gradient descent that maintains moving averages of the gradients and their squares. Its update steps are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Here g_t is the current gradient, m_t is the first-moment estimate (mean), v_t is the second-moment estimate (variance), and ϵ prevents division by zero.

Cross-Entropy Loss

The loss function used is **cross-entropy loss**, which is suitable for multiclass classification. If $\mathbf{y} \in \{0,1\}^C$ is the one-hot label vector and $\hat{\mathbf{y}} \in [0,1]^C$ is the predicted distribution after softmax, the loss is:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

where $C = 17$ is the number of classes. This function strongly penalizes predictions far from the correct class and encourages probability mass to concentrate on the target class.

2.1.2 The Common MLP Block

The final part of this MLP, namely the transition from 64 to 17 neurons, is called the **Common MLP Block** and is reused in later models. It consists of:

- **Dropout** with $p = 0.2$
- A **linear layer** from 64 to 17 neurons

This block maps the model's internal latent representations to probabilities associated with each of the 17 chemical classes. The final output is processed by a `softmax` function, which assigns a normalized probability to each class.

2.2 Encoder MLP

In addition to the simple MLP described in Section 2.1, we tested a slightly more complex model, referred to as the **Encoder MLP** (**EMLP**). It projects the input into a low-dimensional latent space, from which the final prediction is obtained through the Common MLP Block defined in Section 2.1.2.

Motivation

The latent space compresses the information contained in the 16 original features into a more compact representation, potentially helping the model generalize and avoid overfitting. This is especially useful when the number of samples is limited or the features are partly redundant.

Model structure

The overall EMLP architecture consists of two main blocks:

1. **Encoder**: a linear layer that projects the input $\mathbf{x} \in \mathbb{R}^{16}$ into a latent space of dimension d, where $d \in \{4, 8\}$.
2. **Common MLP Block**: as described above, a Dropout + Linear block that projects from 64 units to 17 classes.

The architecture can therefore be represented as:

$$\mathbb{R}^{16} \xrightarrow{\text{Linear}} \mathbb{R}^{d} \xrightarrow{\text{ReLU + Dropout}} \mathbb{R}^{d} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{Common MLP Block}} \mathbb{R}^{17}$$

The two model variants use:

- **EMLPs** (Small): latent dimension $d = 4$
- **EMLPb** (Big): latent dimension $d = 8$

This distinction makes it possible to evaluate how model capacity affects the trade-off between accuracy and generalization.

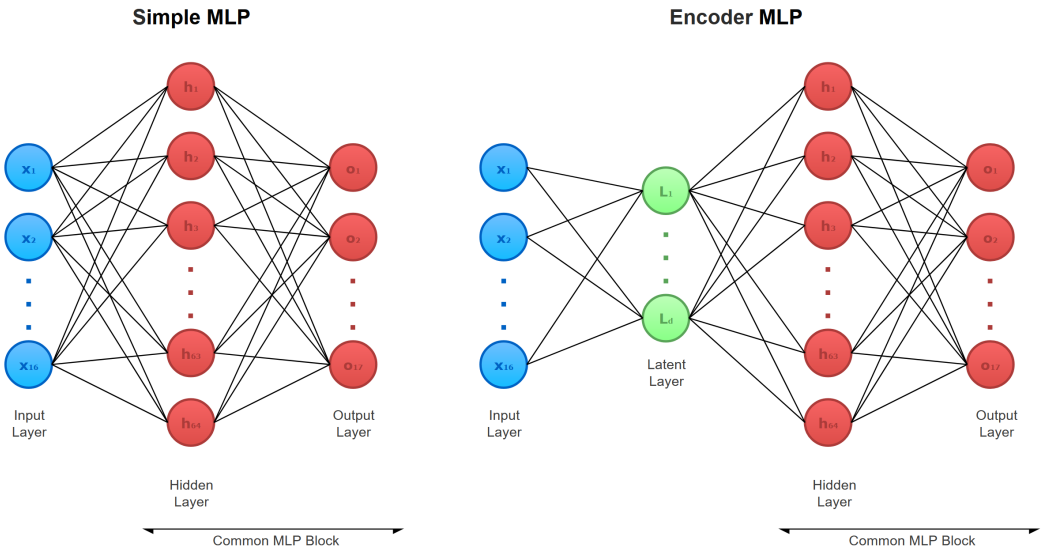


Figure 2.1 Comparison between the simple **MLP** (left) and the **Encoder MLP** (right). The latter introduces a latent space (green) before the *Common MLP Block*.

2.3 Settings shared by all models

All multiclass classification models presented in this and subsequent chapters were trained with the same hyperparameter settings to ensure a rigorous and comparable evaluation of their performance.

- **Learning Rate (LR)**: the learning rate was set to 1×10^{-4}. This parameter controls the size of the optimizer's steps during gradient descent.
- **Weight Decay**: a value of 1×10^{-5} was used to penalize large weights, acting as regularization against overfitting.
- **Maximum number of epochs**: set to **50**. Each epoch is one complete pass over the training dataset.
- **Early Stopping**: early stopping was used with a **patience** of 5 (that is, #Epochs/10), halting training if validation loss did not improve for five consecutive epochs.
- **Dropout**: dropout with probability $p = 0.2$ was introduced to prevent overfitting. It randomly deactivates some units during training, forcing the model not to depend on individual activations.

These settings are maintained across all experiments unless otherwise stated.

2.4 Results and observations

2.4.1 Metric definitions

When evaluating model performance, we distinguish between two levels of accuracy:

- **Sample-level Accuracy - $\mathbf{S_{acc}}$**: the percentage of time instants (individual table rows) classified correctly. It is calculated as:

$$\mathrm{S_{acc}} = \frac{\#\text{ correct samples}}{\#\text{ total samples}}$$

- **Experiment-level Accuracy - $\mathbf{E_{acc}}$**: the percentage of experiments (entire tables) classified correctly through *majority voting* over their samples. For each experiment, the most frequently predicted class across all rows is selected. The formula is:

$$\mathrm{E_{acc}} = \frac{\#\text{ correct experiments}}{\#\text{ total experiments}}$$

Results Table 2.1 reports the results obtained by the MLP and EMLP models on the training, validation, and test sets, at both sample and experiment level. Figures 2.2 and 2.3 show the learning curves and confusion matrices of the best-performing model for this approach: the **MLP**.

Model	Size (MB)	Epochs	Train S_{acc}	Val. S_{acc}	Test S_{acc}	Train E_{acc}	Val. E_{acc}	Test E_{acc}
MLP	0.009	50	0.750	0.636	**0.470**	0.799	0.658	**0.439**
EMLPb	0.007	17	0.469	0.334	0.334	0.475	0.317	0.292
EMLPs	0.006	49	0.438	0.393	0.371	0.478	0.341	0.341

Table 2.1 Model performance with the time-instant approach.

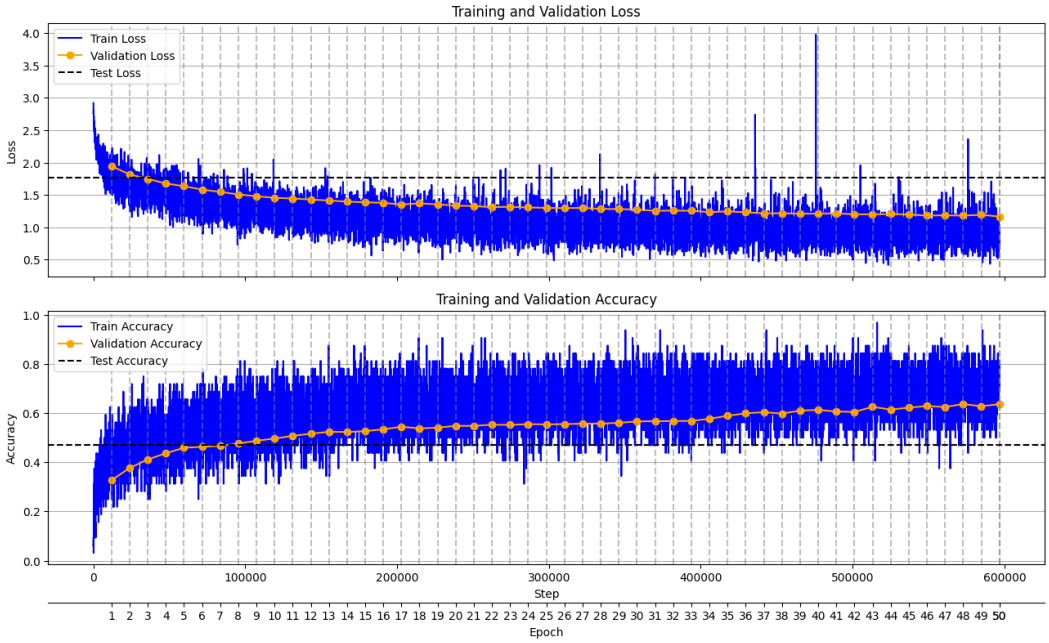


Figure 2.2 Learning curve of the time-instant MLP during training: loss and accuracy on the training and validation sets.

Observations The **MLP** achieved the best performance across all measures, with **47.0%** sample-level accuracy and **43.9%** experiment-level accuracy. Despite being the simplest architecture, it is the most effective in this setting. With very compact inputs (16 features) treated as independent instants, introducing an additional latent space, as in the EMLPs, provides no benefit and may even degrade performance.

Both EMLPs show strong signs of underfitting, with very low accuracy on both training and test data, indicating difficulty learning meaningful structure. This may be caused by limited representational capacity, especially for EMLPs with

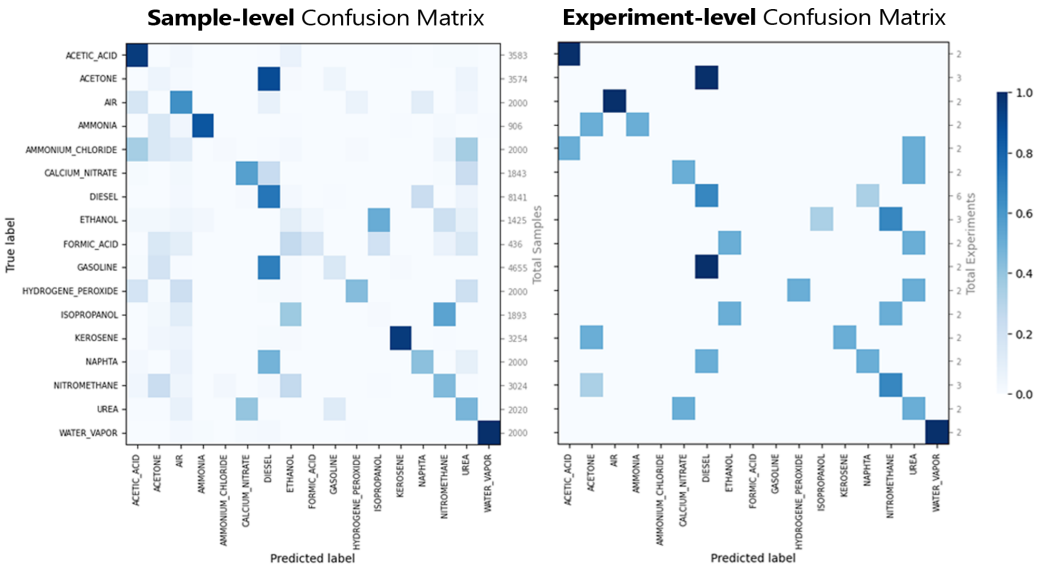


Figure 2.3 Confusion matrices on the test set for the time-instant MLP. Left: *sample-level* classification; right: *experiment-level* classification.

$d = 4$, as well as by the harder optimization problem introduced by an unnecessary intermediate space.

These results confirm that, for the time-instant approach, a simple but well-calibrated architecture is often more effective than more complex models, especially when temporal context is ignored. In such settings, it is preferable to maximize efficiency and direct discriminative capacity rather than seek latent abstractions.

Figure 2.3 shows the confusion matrices for the **MLP** with the time-instant approach at sample level (left) and experiment level (right).

At the **sample level**, the model performs well on clearly distinct classes such as *WATER VAPOR* and *ACETIC ACID*, which show a strong diagonal. Many other classes exhibit substantial dispersion, however, indicating uncertainty or confusion among chemically similar substances. For example, predictions for *AMMONIA*, *ETHANOL*, and *ACETONE* are distributed across several labels, reflecting the similarity of their sensor patterns.

The **experiment-level** matrix, obtained through majority voting over individual instants, shows greater prediction consistency, although some systematic errors remain. The model maintains a degree of accuracy even on experiments with highly variable data, confirming that the voting strategy is robust to local noise.

Chapter 3

Classification with Time Windows

This chapter investigates an alternative approach to sensor-measurement classification: the **time-window approach**. Unlike Chapter 2, where each time instant was treated as an independent sample, here an entire temporal sequence is used as one input unit.

3.1 Data structure: time windows

Each experiment is segmented into fixed-size $N \times 16$ time windows, where $N = 100$. Each window is a training sample consisting of 100 consecutive observations across the 16 available features.

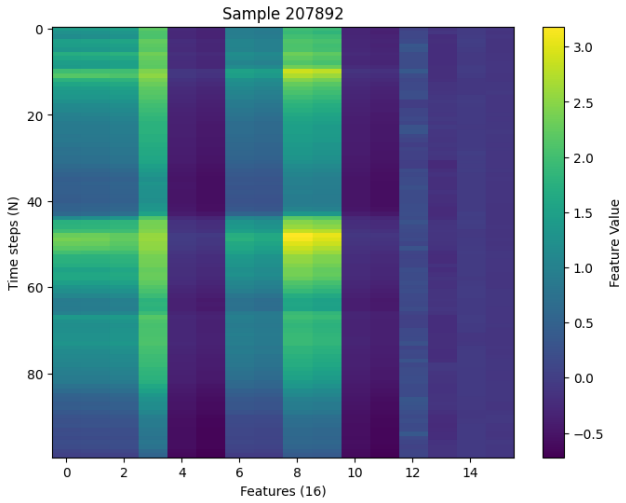


Figure 3.1 Example representation of a 100×16 time-window sample (time steps $\times$ features). Structural differences among columns are clearly visible, indicating the heterogeneous response of the features to the chemical signal.

To maximize the amount of available data, an **overlapping** mechanism was adopted in which each successive window advances by one time step (an overlap of

$N - 1$). This produces many samples from each experiment and improves learning quality.

Each window is assigned the class of the experiment from which it was extracted, preserving semantic consistency.

Models used

All models described in this section share the hyperparameter settings detailed in Section 2.3 and use the *Common MLP Block* from Section 2.1.2 as their final block.

MLP The first model is the simple MLP introduced in Section 2.1, applied to the flattened version of the 100×16 time window. Its architecture is:

$$\mathbb{R}^{100\times16} \xrightarrow{\text{Flatten}} \mathbb{R}^{1600} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{Common MLP Block}} \mathbb{R}^{17}$$

EMLP The EMLP described in Section 2.2 was also used. It includes a compression layer into a latent space of dimension d before projection to 64 intermediate neurons. The two tested variants use $d \in \{20, 80\}$:

$$\mathbb{R}^{100\times16} \xrightarrow{\text{Flatten}} \mathbb{R}^{1600} \xrightarrow{\text{Linear}} \mathbb{R}^{d} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{Common MLP Block}} \mathbb{R}^{17}$$

3.2 Column Selective Encoder MLP

The **CSEMLP** is an EMLP variant in which connections between the input and latent layer are **column selective**. Rather than fully connecting the input vector to the latent neurons, it defines:

- M neurons connected only to the data in each signal column (feature);
- K neurons fully connected to the complete input vector.

The architecture therefore becomes:

$$\mathbb{R}^{100\times16} \xrightarrow{\text{Flatten}} \mathbb{R}^{1600} \xrightarrow{\text{Selective Linear}} \mathbb{R}^{16\cdot M+K} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{Common MLP Block}} \mathbb{R}^{17}$$

Two configurations were tested:

- **CSEMLPs**: $M = 1$, $K = 3$ (19 latent dimensions in total)
- **CSEMLPb**: $M = 5$, $K = 3$ (83 latent dimensions in total)

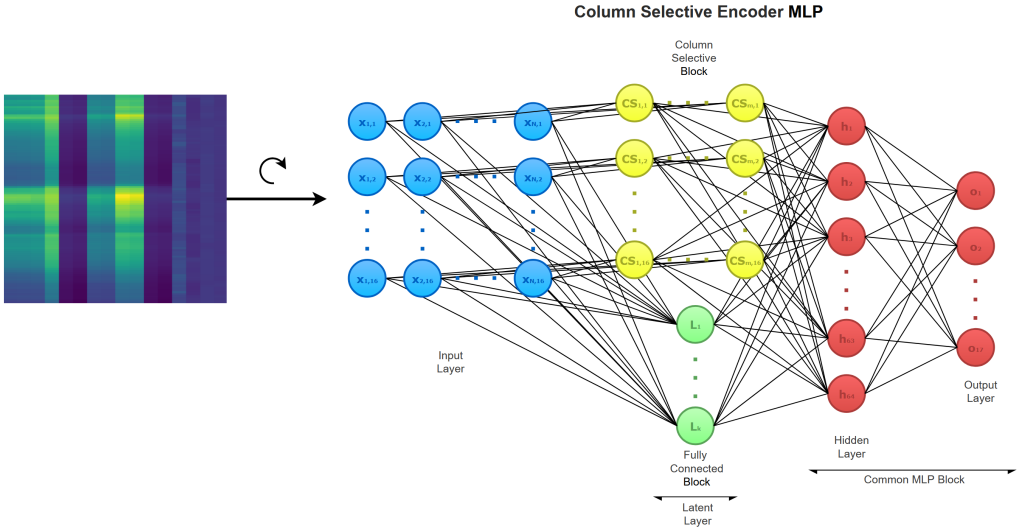


Figure 3.2 Architecture of the **CSEMLP**. The signal is processed by a column-selective block (CS) and a fully connected block (L) before entering the *Common MLP Block*.

3.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are especially effective for processing data with spatial or temporal structure. They apply convolutional filters that learn local, translation-invariant representations. CNNs are widely used across artificial intelligence because they learn hierarchical representations directly from data. Recent studies have highlighted their strong performance in classification, object detection, and semantic segmentation, making them a preferred choice in many modern applications[5].

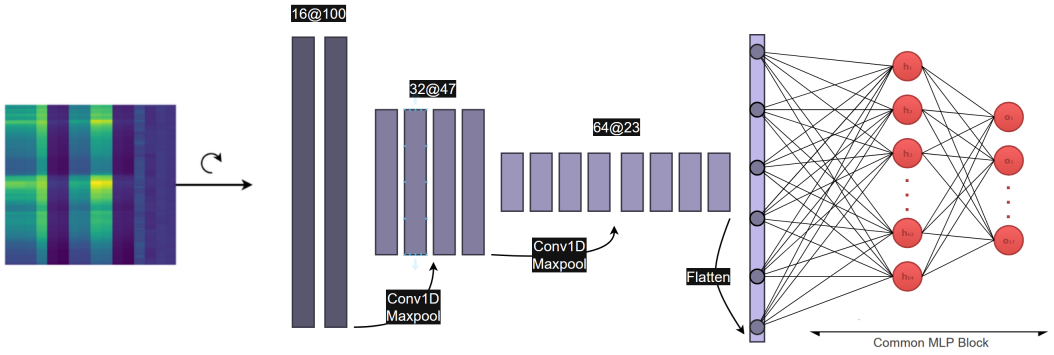


Figure 3.3 Architecture of the **CNN1D**. The 100×16 time sequence is transposed into a 16×100 tensor and processed with 1D convolutions along the temporal axis. The output is then flattened and passed to the *Common MLP Block*.

Convolutional block:

$$\mathbb{R}^{100\times16} \xrightarrow{\text{Transpose}} \mathbb{R}^{16\times100} \xrightarrow{\text{Conv1d}(16\to32,\ \text{kernel}=3)} \mathbb{R}^{32\times98} \xrightarrow{\text{MaxPool1d}(2)+\text{Dropout}} \mathbb{R}^{32\times49}$$

$$\xrightarrow{\text{Conv1d}(32\to64,\ \text{kernel}=3)} \mathbb{R}^{64\times47} \xrightarrow{\text{MaxPool1d}(2)+\text{Dropout}} \mathbb{R}^{64\times23} \xrightarrow{\text{Flatten}} \mathbb{R}^{1472}$$

Fully connected block:

$$\xrightarrow{\text{Linear}(1472\to64)} \mathbb{R}^{64} \xrightarrow{\text{ReLU}+\text{Dropout}} \mathbb{R}^{64} \xrightarrow{\text{Linear}(64\to17)} \mathbb{R}^{17}$$

This architecture exploits the sequential nature of the data and applies 1D convolutions along the temporal axis to model local patterns in multivariate time series.

At the start of the model, two convolutional layers with kernel size 3 operate along the temporal axis to capture local structures and short-term dependencies. Each convolution is followed by *MaxPooling1D* with **kernel size 2**, progressively reducing temporal resolution and computational dimensionality while retaining salient information. This combination extracts increasingly abstract hierarchical representations, which are flattened and passed to the *Common MLP Block* for final classification.

3.4 Results and observations

Results Table 3.1 reports the accuracy of the MLP, EMLP, CSEMLP, and CNN models on the training, validation, and test sets at both sample and experiment level. Figures 3.4 and 3.5 show the learning curves and confusion matrices of the best model for this approach: **EMLPs**.

Model	Size (MB)	Epochs	Train S_{acc}	Val. S_{acc}	Test S_{acc}	Train E_{acc}	Val. E_{acc}	Test E_{acc}
MLP	0.414	10	0.719	0.628	0.481	0.903	0.683	0.536
EMLPs	0.138	30	0.812	0.680	**0.621**	0.885	0.707	**0.683**
EMLPb	0.537	12	0.750	0.696	0.575	0.956	0.780	0.634
CSEMLPs	0.035	9	0.750	0.606	0.501	0.840	0.731	0.585
CSEMLPb	0.077	9	0.688	0.636	0.516	0.882	0.707	0.585
CNN	0.413	9	0.844	0.607	0.563	0.985	0.658	0.682

Table 3.1 Performance of the time-window models in terms of sample-level accuracy (S_{acc}) and experiment-level accuracy (E_{acc}), as defined in Section 2.4.1.

Observations **EMLPs** (Encoder MLP with a 20-dimensional latent space) performs best, achieving the highest sample-level accuracy (**62.1%**) and test experiment-level accuracy (**68.3%**) with a small memory footprint (**0.138 MB**). This configuration offers an excellent compromise between representational capacity and generalization and is well suited to the aggregated temporal structure of the data. Its training accuracy (81.2%) remains moderate relative to more complex models, suggesting effective regularization and abstraction without overfitting.

The **CNN** follows closely with **68.2% experiment-level accuracy**, almost identical to EMLPs, but with a substantially larger memory footprint (**0.413 MB**). Unlike EMLPs, it reaches very high training accuracy (**98.5%**) in only nine epochs, showing a marked tendency to **overfit**. The CNN learns training-set specifics effectively but has greater difficulty generalizing to unseen data.

This is not entirely negative: the CNN's competitive results despite overfitting indicate untapped potential. Stronger regularization (for example, more dropout, more aggressive early stopping, or data augmentation) and a more targeted hyperparameter search over channels, kernel size, stride, and padding could plausibly improve performance, albeit with greater computational complexity than EMLPs.

The **MLP**, although reasonably robust (53.6% at experiment level), is outperformed by encoder-based models, confirming the value of latent-space compression for longer time windows.

The **CSEMLP** models produce interesting results but underperform the standard EMLPs.

Compared with the time-instant results (Section 2.4), sample-level accuracies are similar, but **experiment-level accuracy** is markedly higher with time windows. Windows therefore capture greater temporal coherence and provide a more robust representation of the experiment as a whole.

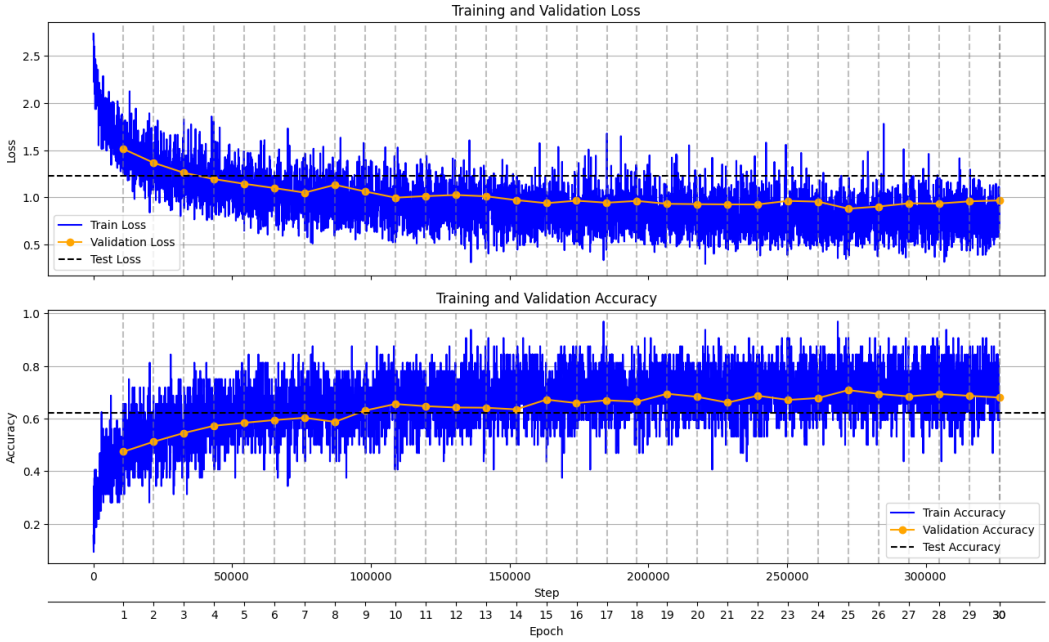


Figure 3.4 Learning curve of EMLPs on time windows during training: loss and accuracy on the training and validation sets.

Figure 3.5 shows the confusion matrices for **EMLPs**, the most effective model for multiclass classification with time windows.

At **sample level**, the matrix has a clear diagonal and excellent performance on many classes, especially *WATER VAPOR*, *ACETIC ACID*, and *UREA*, which are classified correctly in almost all cases. Some confusion remains between chemically related classes such as *ACETONE* and *ETHANOL*, but much less than with

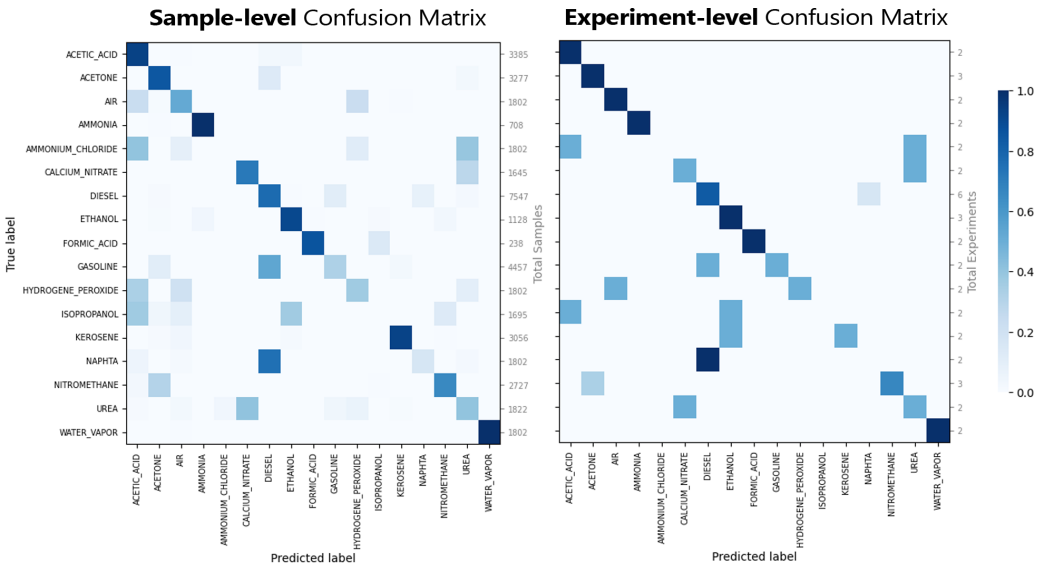


Figure 3.5 Confusion matrices on the test set for EMLPs on time windows. Left: *sample-level* classification; right: *experiment-level* classification.

individual time instants.

The **experiment-level** matrix further confirms the model's robustness: most labels are predicted correctly, with high consistency among instants from the same experiment. The learned representations work locally and remain robust throughout the temporal sequence.

Compared with the MLP confusion matrix for individual instants (Figure 2.3), ambiguity among classes is substantially lower and correct predictions are better distributed. The aggregated temporal information provided by windows is therefore crucial for reliable performance in real-world sensing scenarios.

Both approaches show a recurring tendency to misclassify many classes as **DIESEL**. This can be attributed to the dataset's **intrinsic imbalance**: *DIESEL* contains substantially more experiments and samples than the other classes (Figure 1.1). Its statistical predominance can make models favor this prediction in ambiguous or borderline cases, highlighting the importance of balancing strategies in future work.

Chapter 4

Temporal Analysis of Predictions

This chapter analyzes the **temporal distribution of predictions** generated by the models developed in previous chapters. The goal is to understand how a model's ability to identify the correct class evolves over time, providing information about prediction stability, responsiveness, and consistency.

4.1 Motivation and purpose

With time series, overall accuracy on independent samples is not sufficient; it is also important to observe *when* and *with what confidence* the model recognizes the substance correctly. This analysis can reveal:

- whether the model responds correctly only after a certain exposure time;
- whether predictions fluctuate abnormally during acquisition;
- how stable the prediction remains throughout the experiment (decision continuity);
- whether correctly classified windows or instants cluster in particular temporal regions.

This information is especially useful in real-world applications, where timely and stable responses are essential.

4.2 Results and observations

Results Figure 4.1 visually compares two models: the **MLP** with the *time-instant* approach (left) and **EMLPs** with the *time-window* approach (right). Two representations are shown for each sample in the test experiments:

- **Softmax value for the correct class**: shown as a white-to-blue color map, it indicates the probability assigned to the correct class. Darker blue means greater confidence.

- **Rank of the correct class**: represented by color coding:
  - Green: the correct class is ranked **Top-1**.
  - Yellow: the correct class is within the **Top-5**.
  - Red: the correct class is outside the top five (incorrect prediction).

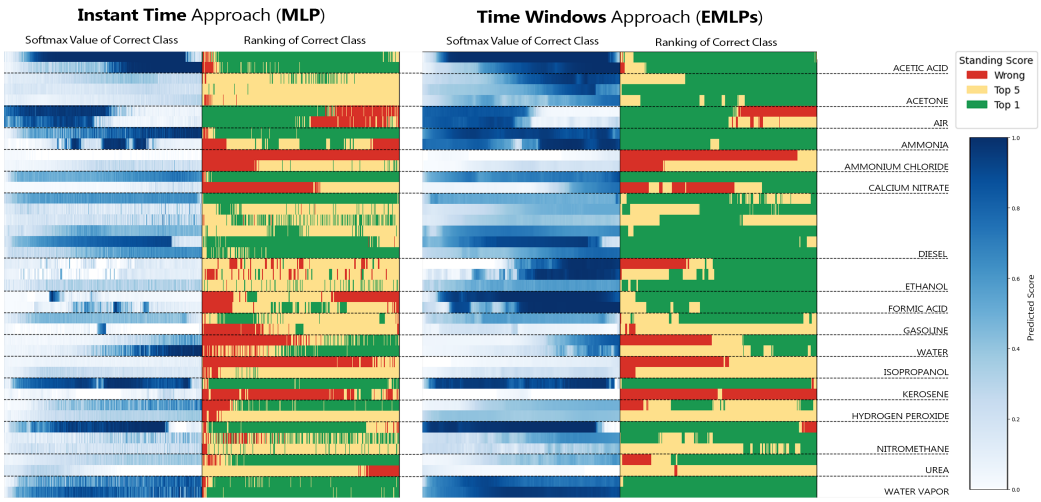


Figure 4.1 Temporal distribution of correct predictions for the two approaches. Left: MLP on individual time instants; right: EMLP on time windows. Each row represents an experiment, ordered by chemical class.

Observations Figure 4.1 clearly shows that the time-window model (EMLPs) is more robust than the time-instant model (MLP). Qualitatively:

- For the **MLP**, softmax values are often low and discontinuous over time, suggesting that predictions are highly sensitive to the individual instant. Red bands in the ranking column reveal frequent uncertainty or failure to recognize the correct class.
- For **EMLPs**, predictions are more temporally consistent and stable, with a more uniform blue color, and the correct class tends to remain in the Top-1 or Top-5 throughout the experiment. This confirms that temporal integration improves the model's overall understanding of the signal.

Chapter 5

Binary Classification Experiments

To investigate the discriminative ability of the previous models, an additional binary classification experiment was conducted on **ETHANOL** and **ACETONE**. These classes were selected because of their chemical similarity and comparable numbers of experiments, making the task challenging but meaningful.

Unlike the main multiclass experiments, the goal is to determine whether the models can capture subtle differences between two classes in a more focused setting. This also enables a more detailed assessment of generalization on a simpler but still practically relevant task.

For each main approach (time instant and time window), the **best model from the preceding chapters** was used:

- The **simple MLP** from Section 2.1, which provided the best accuracy and robustness, was used for the time-instant approach.
- The **EMLPs model** (Encoder + MLP with a 20-dimensional latent space), identified as the best performer in Chapter 3, was used for time windows.

The experiment used **10-fold cross-validation**. In each fold, three experiments per class were assigned to validation, three to test, and the remainder to training. This strategy provides a robust evaluation and reduces variance caused by any single partition.

5.1 Binary classification at individual time instants

Fold	Epochs	Train S_{acc}	Val. S_{acc}	Test S_{acc}	Train E_{acc}	Val. E_{acc}	Test E_{acc}
Fold 1	25	1.000	0.865	0.750	0.984	0.833	0.833
Fold 2	20	1.000	0.784	0.816	1.000	0.833	0.833
Fold 3	18	0.812	0.753	0.843	0.967	0.833	1.000
Fold 4	23	0.935	0.779	0.562	0.951	0.833	0.500
Fold 5	25	0.917	0.589	0.813	0.951	0.500	0.833
Fold 6	18	0.933	0.771	0.751	0.967	0.833	0.667
Fold 7	25	0.846	0.766	0.885	0.967	0.667	1.000
Fold 8	25	0.778	0.886	0.805	0.951	1.000	0.833
Fold 9	25	0.750	0.873	0.771	0.967	1.000	0.833
Fold 10	6	0.857	0.675	0.840	0.852	0.833	1.000
Mean	**20.5**	**0.883**	**0.775**	**0.764**	**0.956**	**0.820**	**0.837**

Table 5.1 Per-fold and mean performance of the MLP on the binary ETHANOL-versus-ACETONE task with the time-instant approach.

5.2 Binary classification with time windows

Fold	Epochs	Train S_{acc}	Val. S_{acc}	Test S_{acc}	Train E_{acc}	Val. E_{acc}	Test E_{acc}
Fold 1	20	0.935	0.968	0.711	1.000	1.000	0.667
Fold 2	6	1.000	0.767	0.843	1.000	0.667	0.833
Fold 3	6	0.889	0.794	0.874	1.000	0.833	0.833
Fold 4	10	1.000	0.898	0.640	1.000	1.000	0.667
Fold 5	10	1.000	0.830	0.812	0.984	0.833	0.833
Fold 6	21	1.000	0.938	0.827	1.000	1.000	0.833
Fold 7	6	1.000	0.781	0.903	1.000	0.833	0.833
Fold 8	7	1.000	0.878	0.956	1.000	0.833	1.000
Fold 9	18	0.923	0.945	0.739	1.000	1.000	0.833
Fold 10	6	1.000	0.673	0.654	1.000	0.667	0.667
Mean	**11.0**	**0.975**	**0.847**	**0.788**	**0.998**	**0.867**	**0.790**

Table 5.2 Per-fold and mean performance of EMLPs on the binary ETHANOL-versus-ACETONE task with the time-window approach.

5.3 Observations

As expected, binary classification improves substantially over the multiclass task, with strong performance from both approaches.

The *time-window approach*, although slightly more complex, performs better at sample level (**78.8%** versus **76.4%**) because it captures local temporal dynamics. Surprisingly, the *time-instant approach* achieves slightly higher mean experiment-level performance (**83.7%** versus **79.0%**). Greater granularity and more predictions for majority voting can therefore sometimes compensate for the lack of temporal context.

This reversal relative to the multiclass case is noteworthy: whereas windows were clearly superior before, both strategies are robust and comparable in the binary setting.

Overall, both approaches generalize effectively and perform consistently across cross-validation folds. This confirms the robustness of the preprocessing pipeline and the effectiveness of the architectures in a less crowded, better-balanced setting.

Chapter 6

Classification with RBF Interpolation

6.1 Approach

This chapter examines an alternative time-series classification approach based on **Radial Basis Functions (RBFs)**. Each time series (one table column, and therefore one sensor feature) is approximated by a linear combination of m radial functions centered at fixed points along the temporal axis.

Formally, a time series $\{y(t_i)\}_{i=1}^{M}$ is treated as a noisy sample of an underlying continuous function $y : \mathbb{R} \to \mathbb{R}$ and approximated as:

$$\tilde{y}(t) = \sum_{j=1}^{m} a_j \, \phi\left(|t - c_j|\right)$$

where:

- ϕ is a radial function, typically Gaussian: $\phi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right)$;
- c_j are the basis centers, equally spaced over the time-series domain;
- a_j are coefficients determined by linear regression.

This approach, previously used successfully for ECG signal recognition [10], drastically reduces dimensionality and improves robustness to noise and local variation.

6.2 Input data structure

Applying this method to each of the 16 time series in an experiment produces a compressed representation with m coefficients per series, for $m \times 16$ features per experiment. With $m = 5$, the resulting vector has dimension:

$$\mathbb{R}^{5\times 16} = \mathbb{R}^{80}$$

This vector represents the entire experiment and serves as input to the classification models.

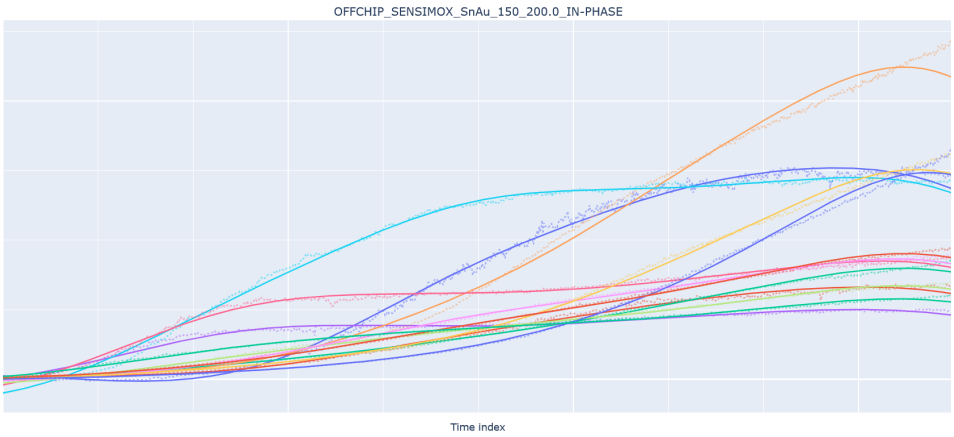


Figure 6.1 Example RBF approximation ($m = 5$) for feature `SnAu_150_200.0_IN-PHASE`. Dashed lines represent original curves from selected `AMMONIUM_CHLORIDE` experiments, while solid lines show the radial-basis fits.

Models used

MLP The following MLP architectures were tested:

$$\mathbb{R}^{80} \to \mathbb{R}^{64} \to \mathbb{R}^{17}, \quad \mathbb{R}^{80} \to \mathbb{R}^{256} \to \mathbb{R}^{17}, \quad \mathbb{R}^{80} \to \mathbb{R}^{64} \to \mathbb{R}^{64} \to \mathbb{R}^{17}$$

All networks use the basic hyperparameters from Section 2.3. Given the static nature of the data, however, the maximum training duration was set to **1000** epochs with **early stopping** and a patience of **100** epochs.

6.3 Random Forest

In addition to neural models, a Random Forest classifier was tested. It is particularly suitable for *tabular* data such as those produced by the RBF approach. Random Forests are ensembles of decision trees trained on random subsets of the data, with a final prediction obtained by majority voting.

Each tree is trained on a random bootstrap sample, while a random feature subset is considered at each node. This introduces diversity and reduces overfitting risk.

Our experiment used 100 estimators (trees).

Random Forests have proved effective in biological and environmental classification, often without complex preprocessing or extensive parameter optimization. In ecology, they have successfully modeled species distributions because they handle complex nonlinear data [11]. They have also achieved notable results in machine-learning competitions, including Kaggle's "Leaf Classification" challenge [8].

6.4 Results and observations

Results

Model	Size (MB)	Epochs	Train Acc.	Val. Acc.	Test Acc.
MLP_64	0.025	598	0.312	0.341	0.268
MLP_256	0.100	716	0.688	0.463	0.390
MLP_64_64	0.042	1000	0.562	0.439	0.414
RandomForest	3.540	–	1.000	**0.658**	**0.439**

Table 6.1 Performance of the different models with the RBF approach.

Observations Table 6.1 shows that the **Random Forest** performs best on validation (**65.8%**) and test data (**43.9%**). Perfect training accuracy (**100%**) is expected because maximum tree depth is set to `None`, allowing complete splitting to purity. This does not necessarily imply overfitting but reflects the model's high capacity to fit the training data.

Among neural networks, **MLP_64_64** performs best, with **41.4%** test accuracy. Reaching the **1000**-epoch limit without early stopping suggests that learning was still in progress and might improve with longer training.

The higher-capacity **MLP_256** performs reasonably well (**39.0%** on test) but tends to overfit slightly. In contrast, **MLP_64** has insufficient capacity and performs poorly across all splits.

Although the Random Forest performs best, its **3.54 MB memory footprint** is substantially larger than those of the neural networks and may constrain deployment on embedded or edge devices.

The best RBF result (**RF: 43.9%**) and the two-layer MLP are comparable to the **time-instant MLP** (also **43.9%**) but substantially below the **time-window approach**, where **EMLPs** achieved **68.3%** experiment-level test accuracy. This confirms the value of temporal information for sensor-signal classification.

Chapter 7

Classification with Continuous Temporal Processing

7.1 Recurrent Neural Networks

This chapter introduces continuous temporal processing, a paradigm particularly suited to sequential data such as time series. It assumes that information acquired at each instant is related to previous instants and that the sequence itself is the entity to be modeled. Unlike models that treat observations independently, recurrent models use internal mechanisms to "remember" past information.

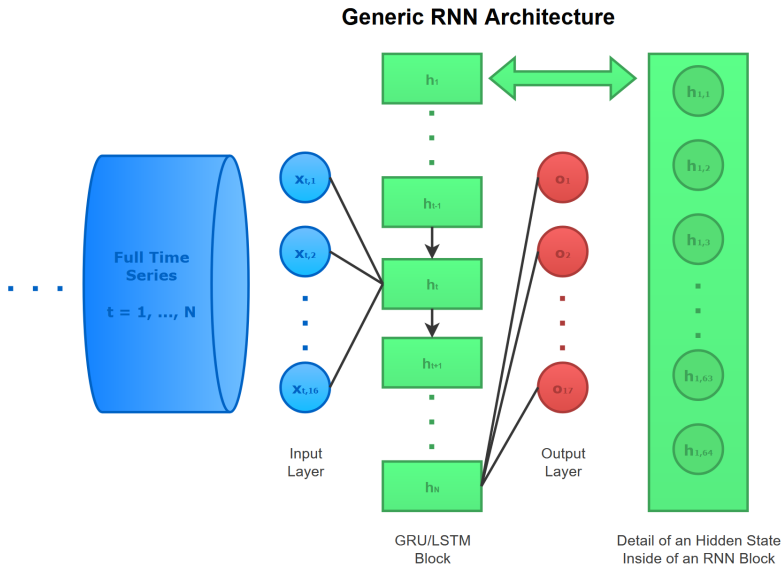


Figure 7.1 Architecture of a generic RNN. Vector x_t connects to hidden state h_t, which in turn connects to h_{t-1} and h_{t+1}. The output layer communicates only with the final hidden state h_N.

Recurrent Neural Networks (RNNs; Figure 7.1) enable sequential processing while maintaining an internal state that is updated at every time step. They are therefore well suited to time-series classification, where signal dynamics contain

information crucial to prediction.

7.2 Gated Recurrent Unit Networks

The Gated Recurrent Unit (GRU) extends classical RNNs and is designed to alleviate vanishing gradients when learning long sequences. Each GRU unit manages two main gates:

- **Reset gate** r_t: determines how much of the previous hidden state h_{t-1} to forget.
- **Update gate** z_t: determines how much of the candidate state $\tilde{h}_t$ is used to update the current state.

The GRU is governed by the following equations:

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), \quad r_t = \sigma(W_r x_t + U_r h_{t-1})$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot ht - 1)), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

where x_t is the input at time t, h_t is the hidden state, and σ is the sigmoid function. Gates z_t and r_t control information flow, enabling more stable learning even on long sequences.

7.3 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks are a more expressive RNN generalization with **explicit long-term memory**, represented by cell state c_t. Each LSTM unit has three main gates:

- **Input gate** i_t: controls how much the current input affects the cell state.
- **Forget gate** f_t: determines how much of the previous state c_{t-1} is retained.
- **Output gate** o_t: controls how much of cell state c_t is emitted as output h_t.

The complete operation is described by:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$
$$\tilde{c}t = \tanh(W_c x_t + U_c ht - 1 + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$$

Unlike GRUs, LSTMs clearly separate the *hidden state* h_t from the *cell memory* c_t, providing greater flexibility for long-term memory at the cost of more parameters and greater computational complexity.

Differences between GRUs and LSTMs. Although LSTMs are more expressive, GRUs are efficient and architecturally simpler. In many practical settings, they achieve comparable performance with fewer parameters and more stable training, especially on moderately sized datasets such as the one studied here.

7.4 Classification of Complete Variable-Length Time Series

Each experiment is treated as a complete sequence while preserving variation in duration. The GRU/LSTM receives $(x_1, x_2, \ldots, x_N)$, where N is variable. Each x_t is processed sequentially, updating a hidden state that summarizes information up to that point:

$$h_t = \text{RNN}(x_t, h_{t-1})$$

At the end of the sequence, h_N is used to classify the entire experiment. This allows the model to learn a dynamic representation of the full signal and capture long-range dependencies.

Hyperparameters match Section 2.3, except for a maximum of 1000 epochs and early-stopping patience of 100. This matches the RBF approach (Chapter 6) because the inputs represent whole experiments.

7.4.1 Results and Observations

Model	Size (MB)	Epochs	Train E_{acc}	Val E_{acc}	Test E_{acc}
GRU	0.067	1000	0.667	0.530	**0.390**
LSTM	0.088	287	0.430	0.368	0.268

Table 7.1 Performance of GRU and LSTM models on complete sequences.

The GRU clearly outperforms the LSTM. With test accuracy of **0.390**, it approaches the best MLP under the RBF approach (0.414).

The GRU remains well below **EMLPs** with time windows, which achieved **0.683** experiment-level test accuracy. Although the GRU models complete variable-length sequences effectively, spatial-temporal compression through latent windows remains superior.

The GRU is also more stable during training. The LSTM stopped early after 287 epochs, indicating more difficult optimization and greater overfitting risk. This confirms the suitability of GRUs for moderately sized datasets with noisy signals.

7.5 Applying the RNN to Time Windows

This section evaluates a GRU on fixed-length time windows ($N = 100$), as in Chapter 3. The goal is to test whether temporal memory improves window classification over the earlier MLPs. The experiment-level prediction is obtained by majority voting over window predictions.

7.5.1 Results and Observations

The GRU on fixed-length windows ($N = 100$), with experiment-level majority voting, yields several useful observations. The confusion matrices and prediction visualiza-

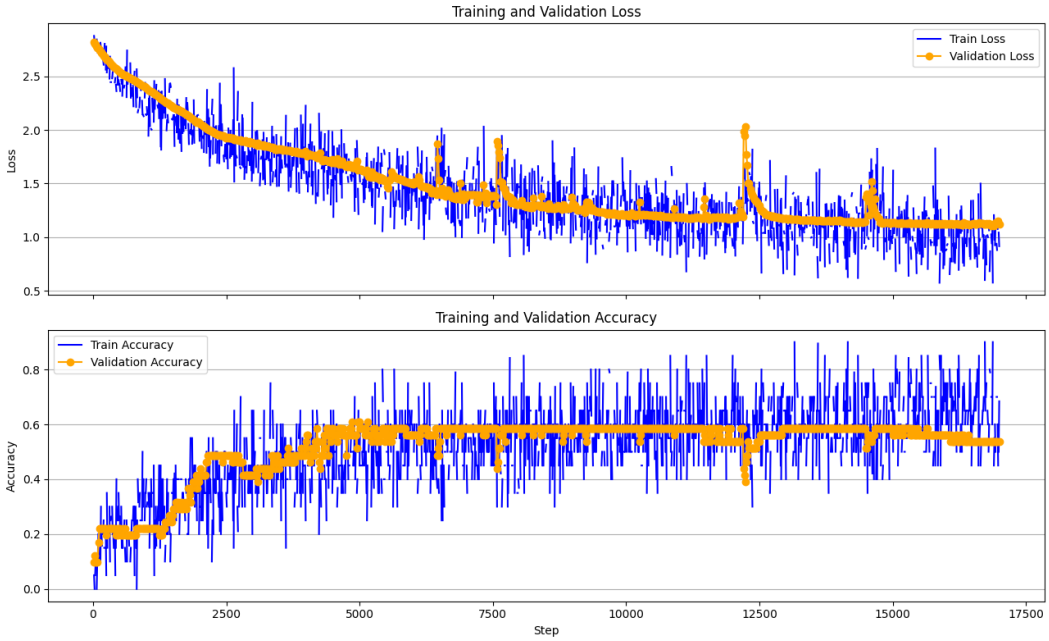


Figure 7.2 Learning curves of the GRU model on variable-length time series.

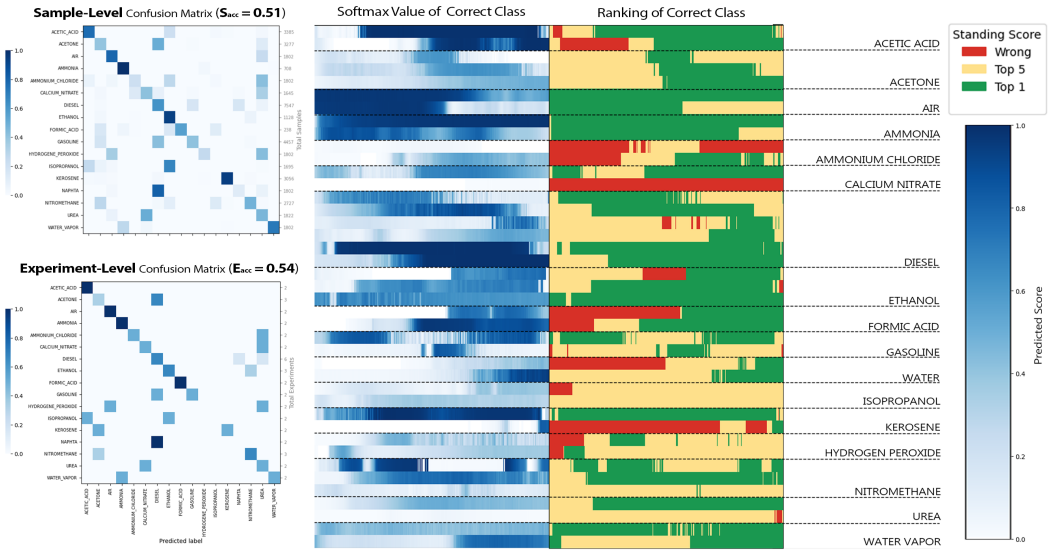


Figure 7.3 Results of the GRU with the time-window approach. Left: sample-level and experiment-level confusion matrices; right: the associated temporal distributions, analogous to Figure 4.1.

tions (Figure 7.3) show solid overall performance, with $E_{acc} = 0.54$, exceeding many window-level MLPs.

The GRU distinguishes `KEROSENE`, `AIR`, and `NITROMETHANE` particularly well, likely because their temporal patterns are more distinctive or repetitive. It struggles with `CALCIUM NITRATE` and `AMMONIUM CHLORIDE`, where memory may introduce more ambiguity than useful information. Temporal modeling can emphasize signifi-

cant variation for some classes but is more fragile when discrimination depends on stationary or local details.

Compared with **EMLPs**, which reached $E_{acc} = 0.683$, the windowed GRU is less effective, although it performs comparably to the **MLP** (0.536).

Chapter 8

Comparative Discussion of the Models

After presenting the individual approaches, this chapter summarizes the main results through a cross-model comparison of accuracy and parameter efficiency. The objective is to highlight trade-offs among architectural complexity, memory footprint, and chemical-classification performance.

Comparison of Performance and Complexity

Figure 8.1 shows the **relationship between model size (MB)** and **experiment-level accuracy** for the four approaches: *time instants*, *time windows*, *RBF approximation*, and *continuous temporal processing* (RNN). Each point is a model colored by approach; horizontal lines identify size outliers omitted from the scatter plot for readability.

Comparative Observations

Figure 8.1 supports several key observations:

- **Time-window approach** — The green models dominate the upper part of the plot and provide the best compromise between accuracy and size. **EMLPs_win** achieves **68.3%** test accuracy with a footprint of 0.138 MB. The more compact **CSEMLP** models also perform well.
- **Time-instant approach** — The blue models are extremely lightweight but sacrifice performance. The simple **MLP** achieves **43.9%** test accuracy at only 9 KB, making it suitable for embedded settings but less competitive overall.
- **RBF approach** — The orange models achieve reasonable results with low complexity. **MLP_64_64** reaches **41.4%** at only 42 KB. The **Random Forest** has similar performance (43.9%) but is a memory outlier at **3.54 MB**.
- **RNN approach** — The red full-sequence models occupy an intermediate region. The 0.067 MB **GRU** reaches **39.0%** test accuracy without surpassing

the window models. The more complex **LSTM** (0.088 MB) reaches only 26.8%. **GRU_win** lies near the CSEMLPs, demonstrating architectural flexibility.

- **Size outliers** — The **Random Forest** (3.54 MB) has performance comparable to some MLPs (43.9%) but a much larger footprint. **MLP_win** (0.414 MB) and **MLPb_win** (0.537 MB) are also disproportionately large relative to their gains. The **CNN** (**0.413 MB**) approaches the upper complexity limit but, unlike the other outliers, has room for improvement: it nearly matches EMLPs and may benefit from better hyperparameter optimization. A well-tuned CNN is therefore a viable, though more expensive, option when accuracy takes priority over maximum compression.

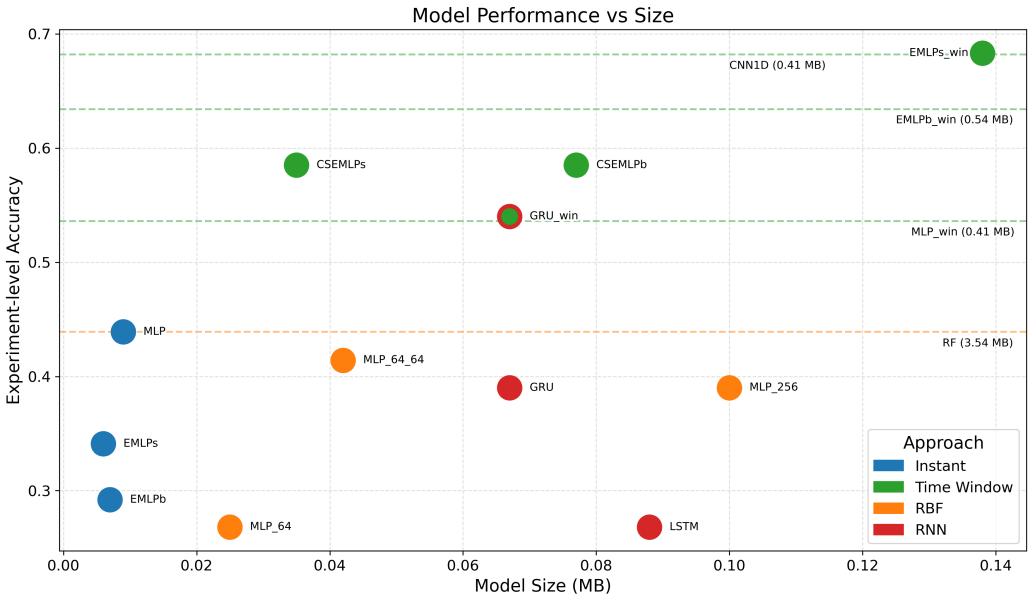


Figure 8.1 Relationship between experiment-level accuracy and model memory footprint.

Final Remarks

The **time-window approach** is the most competitive in both accuracy and efficiency, using compact, well-designed models. **RBF** is a viable alternative for low-compute settings, while RNNs are promising because they model the complete temporal dynamics and may become more robust with additional data.

The confusion matrices also show that models favor classes that are better represented in the training data and penalize underrepresented classes. Future work should balance the dataset by increasing the number of experiments for rare classes, improving generalization and classification fairness.

Overall, temporal information is central to sensor-signal classification. Depending on memory constraints and accuracy priorities, a designer can choose between ultra-compact solutions (MLP) and more sophisticated, higher-performing models (EMLPs_win, GRU_win, or CNN).

Conclusions

This work explored several classification strategies for complex sensor data supplied by *Sensichips S.r.l.*, with the objective of identifying chemical substances from noisy multivariate signals collected in real-world environments. Four main approaches were developed and compared:

- The simple, highly compact **time-instant approach** showed that lightweight architectures can achieve satisfactory results, with 43.9% experiment-level test accuracy.
- The **time-window approach** proved most effective because it captures local signal dynamics: **EMLPs_win** achieved the best experimental performance, with 68.3% accuracy.
- The **RBF approach**, based on compressed time-series representations, performed well especially with MLPs while maintaining high parameter efficiency.
- **Continuous temporal processing** with RNNs (GRU and LSTM) modeled each complete experimental sequence as a coherent temporal stream. The GRU showed promising behavior, particularly when combined with time windows.

Systematic performance analysis clearly demonstrated the importance of temporal information and input representation. Time windows were the most robust approach, while RNNs offer an alternative that exploits complete sequential structure and retains potential for future development.

The confusion matrices and qualitative analysis also showed that models favor better-represented classes. Future work should address class imbalance more carefully to reduce prediction bias and improve generalization.

This study demonstrates that combining architectural design, statistical analysis, and domain understanding can produce effective and interpretable models even in realistic settings.

All source code developed for this thesis, including preprocessing, training, and analysis notebooks, is publicly available in the following GitHub repository:

`https://github.com/mich1803/sensor-ts-classification`

Bibliography

[1] Jiayue Han, Huizi Li, Jiangong Cheng, Xiang Ma, and Yanyan Fu. Advances in metal oxide semiconductor gas sensor arrays based on machine learning algorithms. *J. Mater. Chem. C*, 13:4285–4303, 2025.

[2] Sensichips S.r.l. sensichips.com/air-sensor, 2025. Accessed: 2025-05-21.

[3] Emran Alotaibi and Nadia Nassif. Artificial intelligence in environmental monitoring: in-depth analysis. *Discover Artificial Intelligence*, 4(1):84, 2024.

[4] Adebimpe Ige, Peter Adepoju, and Anfo Pub. Machine learning in industrial applications: An in-depth review and future directions. *International Journal of Multidisciplinary Research and Growth Evaluation*, 06:36–44, 12 2024.

[5] Xia Zhao, Limin Wang, Yufei Zhang, Xuming Han, Muhammet Deveci, and Milan Parmar. A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, 57(4):99, 2024.

[6] Cesar F. Caiafa, Zhe Sun, Toshihisa Tanaka, Pere Marti-Puig, and Jordi Solé-Casals. Special issue "machine learning methods for biomedical data analysis". *Sensors*, 23(23):9377, 2023.

[7] Usman Yaqoob and Mohammad I. Younis. Chemical gas sensors: Recent developments, challenges, and the potential of machine learning. *Sensors*, 21(8), 2021.

[8] Ivan Sosnovik. Leaf classification competition: 1st place winner's interview, 2017. Accessed: 2025-05-21.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*, chapter 10, pages 373–431. MIT Press, Cambridge, MA, 2016.

[10] A. De Gaetano, S. Panunzi, F. Rinaldi, A. Risi, and M. Sciandrone. A patient adaptable ecg beat classifier based on neural networks. *Applied Mathematics and Computation*, 213(1):243–249, 2009.

[11] Richard D Cutler, Thomas C Edwards Jr, Karen H Beard, Adele Cutler, Kyle T Hess, Jonathan Gibson, and Joshua J Lawler. Random forests for classification in ecology. *Ecological applications*, 17(1):81–94, 2007.

[12] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

Acknowledgments

I thought for a long time about whether to write this page, but in the end it felt right to do so. This thesis is, in a sense, the culmination of an intense and stimulating journey, and I believe it is important to acknowledge those who accompanied me along the way.

I would like to divide these acknowledgments into two parts: academic and personal.

Academically, my first thanks go to my classmates Andrea, Francesco C., Julian, Leo, and Francesco M. Your presence made every day in Rome lighter and every lesson more enjoyable. Special thanks go to Francesco M., with whom I had the pleasure of collaborating on several projects this year and with whom I hope to continue working.

My sincere thanks go to the professors who guided us and conveyed enthusiasm as well as knowledge. In particular, I thank Professor Puppo for creating the degree program in "Mathematical Sciences for Artificial Intelligence," which immediately appealed to me by combining one of my passions with technological innovation. I also thank Professor Sciandrone, the advisor of this thesis, for his availability and for the opportunity he offered me.

Finally, I thank the research group of Chris, Laura, Elisa, and Gabriele, who welcomed Francesco and me and guided us through a research experience that I never imagined I would find so fascinating.

On a personal level, I am deeply grateful to my family for their support and patience throughout this journey. Even in moments of fatigue or uncertainty, I always felt the reassurance of their steady support in everyday life.

Special thanks go to Cristina, who has been by my side since my final year of high school. Your quiet but constant presence made all the difference.

Lastly, I also wish to acknowledge those who, despite not following my university journey closely, were present at important moments. Thank you for being there, each in your own way.